

尚硅谷大数据技术之 Redis

（作者：尚硅谷研究院）

第 1 章 NoSQL

1.1 NoSQL 数据库

1.1.1 NoSQL 是什么

- （1）NoSQL(Not Only SQL)，意即“不仅仅是 SQL”，泛指非关系型的数据库。
- （2）NoSQL 不拘泥于关系型数据库的设计范式，放弃了通用的技术标准，为某一领域特定场景而设计，从而使性能、容量、扩展性都达到了一定程度的突破。

1.1.2 NoSQL 的特点

- （1）不遵循 SQL 标准
- （2）不支持 ACID
- （3）远胜于 SQL 的性能。

1.1.3 NoSQL 的适用场景

- （1）对数据高并发的读写
- （2）海量数据的读写
- （3）对数据高可扩展性的

1.1.4 NoSQL 的不适用场景

- （1）需要事务（ACID）支持
- （2）基于 sql 的结构化查询存储，处理复杂的关系,需要即席查询。
- （3）用不着 sql 的和用了 sql 也不行的情况，请考虑用 NoSql

1.2 NoSQL 家族

1) Memcached

- （1）很早出现的 NoSQL 数据库
- （2）数据都在内存中，一般不持久化

(3) 支持简单的 key-value 模式，数据类型支持单一

(4) 一般是作为缓存数据库辅助持久化的数据库

2) Redis

(1) 几乎覆盖了 Memcached 的绝大部分功能

(2) 数据都在内存中，支持持久化，主要用作备份恢复

(3) 支持丰富的数据类型，例如 string、list、set、zset、hash 等

(4) 一般是作为缓存数据库辅助持久化的数据库

3) mongoDB

(1) 高性能、开源、模式自由的文档型数据库

(2) 数据都在内存中，如果内存不足，把不常用的数据保存到硬盘

(3) 虽然是 key-value 模式，但是对 value(尤其是 json)提供了丰富的查询功能

(4) 支持二进制数据及大型对象

(5) 可以根据数据的特点替代 RDBMS(关系数据库管理系统)，成为独立的数据库。

或者配合 RDBMS，存储特定的数据

4) HBase

(1) Hbase 是 Hadoop 项目的数据库，主要用于对大量数据进行随机、实时的读写操作。

(2) Hbase 能支持到数十亿行 × 百万列的数据表

5) Cassandra

(1) Cassandra 用于管理由大量商用服务器构建起来的庞大集群上的海量数据集(PB 级)

6) Neo4j

(1) Neo4j 是基于图结构的数据库，一般用于构建社交网络、交通网络、地图等

第 2 章 Redis 简介

2.1 Redis 官网

(1) Redis 官方网站 <http://redis.io>

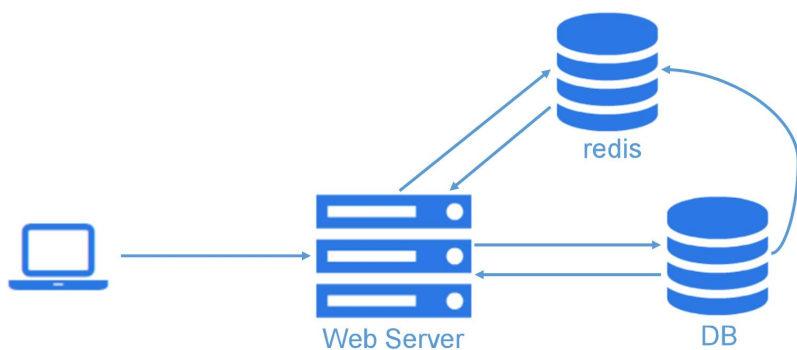
2.2 Redis 是什么

- (1) Redis 是一个开源的 key-value 存储系统。
- (2) 它支持存储的 value 类型相对更多，包括 string(字符串)、list(链表)、set(集合)、zset(sorted set)和 hash (哈希类型)。
- (3) Redis 会周期性的把更新的数据写入磁盘或者把修改操作写入追加的记录文件
- (4) 支持高可用和集群模式。

2.3 Redis 的应用场景

2.3.1 配合关系型数据库做高速缓存

- (1) 高频次，热门访问的数据，降低数据库 IO
- (2) 经典的 Cache Aside Pattern (旁路缓存模式)

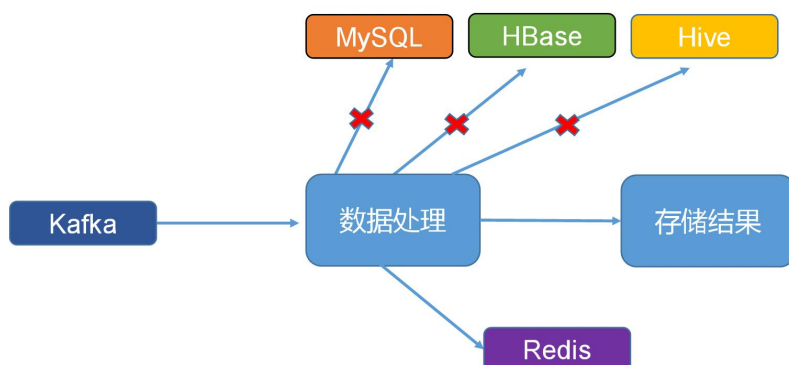


让天下没有难学的技术

2.3.2 大数据场景

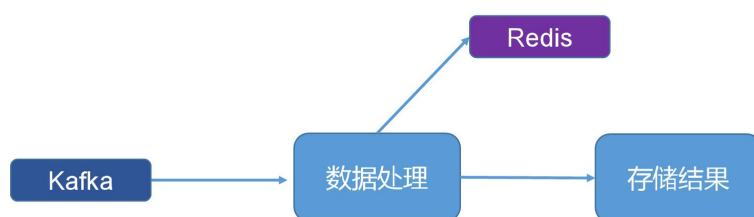
1) 缓存数据

- (1) 需要高频次访问
- (2) 持久化数据访问较慢



2) 临时数据

- (1) 高频次
- (2) 读写时效性高
- (3) 总数据量不大
- (4) 临时性
- (5) 用 key 查询



3) 计算结果

- (1) 高频次写入
- (2) 高频次查询

(3) 总数据量不大



让天下没有难学的技术

2.3.3 利用其多样的数据结构存储特定的数据

- (1) 最新 N 个数据 → 通过 List 实现按自然事件排序的数据
- (2) 排行榜, TopN → 利用 zset(有序集合)
- (3) 时效性的数据, 比如手机验证码 → Expire 过期
- (4) 计数器, 秒杀 → 原子性, 自增方法 INCR、DECR
- (5) 去除大量数据中的重复数据 → 利用 set 集合
- (6) 构建队列 → 利用 list 集合
- (7) 发布订阅消息系统 → pub/sub 模式

第 3 章 Redis 安装

3.1 安装

1) 安装新版 gcc 编译器

```
[atguigu@hadoop102 ~]$ sudo yum -y install gcc-c++
```

2) 上传 redis-6.2.1.tar.gz 安装包到/opt/software 目录下

3) 解压 redis-6.2.1.tar.gz 到/opt/module 目录下

```
[atguigu@hadoop102 software]$ tar -zxvf redis-6.2.1.tar.gz -C /opt/module/
```

4) 之后进入安装包的 src 目录, 编辑 Makefile 文件, 修改软件安装路径如下:

```
[atguigu@hadoop102 src]$ vim Makefile
```

#修改如下

```
PREFIX?=/home/atguigu
```

5) 在 Redis 的解压路径下执行编译和安装命令

```
[atguigu@hadoop102 redis-6.2.1]$ make && make install
```

3.2 查看安装目录 /home/atguigu/bin

(1) redis-benchmark:性能测试工具,可以在自己本子运行,看看自己本子性能如何
(服务启动起来后执行)

(2) redis-check-aof: 修复有问题的 AOF 文件

(3) redis-check-dump: 修复有问题的 RDB 文件

(4) redis-sentinel: 启动 Redis 哨兵服务

(5) redis-server: Redis 服务器启动命令

(6) redis-cli: 客户端,操作入口

3.3 Redis 的启动

1) 拷贝一份 redis.conf 配置文件到工作目录

```
[atguigu@hadoop102 ~]$ mkdir myredis
[atguigu@hadoop102 ~]$ cd myredis
[atguigu@hadoop102 myredis]$ cp /opt/module/redis-6.2.1/redis.conf .
```

2) 绑定主机 IP, 修改 bind 属性

```
[atguigu@hadoop102 myredis]$ vim redis.conf
bind 0.0.0.0
```

3) 指定配置文件进行启动

```
[atguigu@hadoop102 myredis]$ redis-server redis.conf
```

3.4 客户端访问

1) 使用 redis-cli 命令访问启动好的 Redis, 默认端口为 6379

```
[atguigu@hadoop102 myredis]$ redis-cli
127.0.0.1:6379>
```

2) 如果有多个 Redis 同时启动, 或者端口做了修改, 则需指定端口号访问

```
[atguigu@hadoop102 myredis]$ redis-cli -p 6379
127.0.0.1:6379>
```

3) 如果访问非本机 redis, 需要指定 host 来访问

```
[atguigu@hadoop102 myredis]$ redis-cli -h 127.0.0.1 -p 6379
127.0.0.1:6379>
```

4) 通过 ping 命令测试验证

```
127.0.0.1:6379> ping
PONG
```

3.5 关闭 Redis 服务

如果还未通过客户端访问，可直接 `redis-cli shutdown`

```
[atguigu@hadoop102 myredis]$ redis-cli shutdown
```

如果已经进入客户端,直接 `shutdown` 即可.

```
127.0.0.1:6379> shutdown
```

第 4 章 Redis 的五大数据类型

4.1 帮助手册

<http://redisdoc.com/>

4.2 redis 键(key)

1) 查看当前库的所有键

```
127.0.0.1:6379> keys *
```

2) 判断某个键是否存在

```
127.0.0.1:6379> exists <key>
```

3) 查看键对应的 value 的类型

```
127.0.0.1:6379> type <key>
```

4) 删除某个键

```
127.0.0.1:6379> del <key>
```

5) 设置过期时间

```
127.0.0.1:6379> expire <key> <second>
```

6) 查看过期时间, -1 表示永不过期, -2 表示已过期

```
127.0.0.1:6379> ttl <key>
```

7) 查看当前库中 key 的数量

```
127.0.0.1:6379> dbsize
```

8) 清空当前库

```
127.0.0.1:6379> flushdb
```

9) 清空所有库

```
127.0.0.1:6379> flushall
```

4.3 String

4.3.1 特点

(1) String 是 Redis 最基本的类型, 适合保存单值类型, 即一个 key 对应一个 value

(2) String 类型是二进制安全的。意味着 Redis 的 string 可以包含任何数据。比如 jpg 图片或者序列化的对象。

(3) 一个 Redis 中字符串 value 最多可以是 512M

4.3.2 常用操作

1) 添加键值对

```
127.0.0.1:6379> set <key> <value>
```

2) 获取键的值

```
127.0.0.1:6379> get <key>
```

3) 将给定的<value>追加到原值的末尾

```
127.0.0.1:6379> append <key> <value>
```

4) 获取值的长度

```
127.0.0.1:6379> strlen <key>
```

5) 当 key 不存在时设置 key 的值

```
127.0.0.1:6379> setnx <key> <value>
```

6) 将 key 中存储的数字值增 1

```
127.0.0.1:6379> incr <key>
```

7) 将 key 中存储的数字值减 1

```
127.0.0.1:6379> decr <key>
```

8) 将 key 中存储的数字值按照指定步长增

```
127.0.0.1:6379> incrby <key> <步长>
```

9) 将 key 中存储的数字值按照指定步长减

```
127.0.0.1:6379> decrby <key> <步长>
```

10) 同时添加一个或者多个 key

```
127.0.0.1:6379> mset <k1> <v1> <k2> <v2>
```

11) 同时获取一个或者多个 key 的值

```
127.0.0.1:6379> mget <k1> <k2> <k3>
```

12) 同时添加一个或者多个 key，当且仅当所有给定的 key 都不存在

```
127.0.0.1:6379> msetnx <k1> <v1> <k2> <v2>
```

13) 获取值的子串

```
127.0.0.1:6379> getrange <key> <start> <end>
```

14) 从指定的开始位置覆盖旧值

```
127.0.0.1:6379> setrange <key> <start> <value>
```

15) 同时设置值和过期时间

```
127.0.0.1:6379> setex <key> <seconds> <value>
```

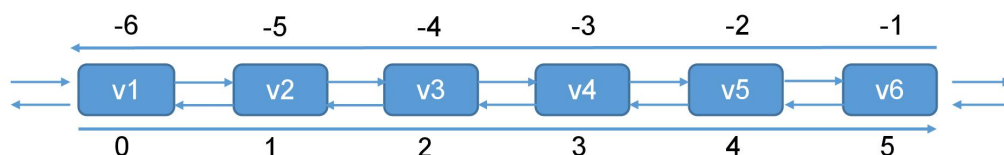
16) 设置新值的同时获取旧值

```
127.0.0.1:6379> getset <key> <value>
```


4.4 List

4.4.1 特点

- (1) 单键多值
- (2) Redis List 是简单的字符串列表，按照插入顺序排序。你可以添加一个元素到列表的头部（左边）或者尾部（右边）。
- (3) 它的底层实际是个双向链表，对两端的操作性能很高，通过索引下标的操作中间的节点性能会较差



让天下没有难学的技术

4.4.2 常用操作

1) 从左边插入一个或多个值

```
127.0.0.1:6379> lpush <key> <element...>
```

2) 从右边插入一个或多个值

```
127.0.0.1:6379> rpush <key> <element...>
```

3) 从左边删除一个值（值在键在，值光键亡）

```
127.0.0.1:6379> lpop <key>
```

4) 从右边删除一个值（值在键在，值光键亡）

```
127.0.0.1:6379> rpop <key>
```

5) 从 key1 列表右边删除一个值，插入到 key2 列表左边

```
127.0.0.1:6379> rpoplpush <key1> <key2>
```

6) 按照索引下标范围获取元素（从左到右）

```
127.0.0.1:6379> lrange <key> <start> <stop>
```

7) 按照索引下标获取元素（从左到右）

```
127.0.0.1:6379> lindex <key> <index>
```

8) 获取列表长度

```
127.0.0.1:6379> llen <key>
```

9) 在指定<value>的前面或者后面插入<newvalue>

```
127.0.0.1:6379> linsert <key> before|after <pivot> <element>
```

10) 从左边删除 count 个指定的 value

```
127.0.0.1:6379> lrem <key> <count> <element>
```

4.5 Set

4.5.1 特点

(1) set 中的元素是无序不重复的，当你需要存储一个列表数据，又不希望出现重复数据时，set 是一个很好的选择，并且 set 提供了判断某个成员是否在一个 set 集合内的重要接口，

(2) Redis 的 Set 是 string 类型的无序集合。它底层其实是一个 value 为 null 的 hash 表，所以添加，删除，查找的复杂度都是 O(1)。

4.5.2 常用操作

1) 将一个或者多个 member 元素加入到集合中，已经存在的 member 将被忽略

```
127.0.0.1:6379> sadd <key> <member...>
```

2) 取出集合的所有值

```
127.0.0.1:6379> smembers <key>
```

3) 判断集合<key>是否包含指定的 member，包含返回 1，不包含返回 0

```
127.0.0.1:6379> sismember <key> <member>
```

4) 返回集合的元素个数

```
127.0.0.1:6379> scard <key>
```

5) 从集合中删除指定的元素

```
127.0.0.1:6379> srem <key> <member...>
```

6) 随机从集合中删除一个值

```
127.0.0.1:6379> spop <key>
```

7) 随机从集合中取出 n 个值，不会从集合中删除

```
127.0.0.1:6379> srandmember <key> <count>
```

8) 返回多个集合的交集元素

```
127.0.0.1:6379> sinter <key...>
```

9) 返回多个集合的并集元素

```
127.0.0.1:6379> sunion <key...>
```

10) 返回多个集合的差集元素

```
127.0.0.1:6379> sdiff <key...>
```

4.6 Zset

4.6.1 特点

(1) Redis 有序集合 zset 与普通集合 set 非常相似，是一个没有重复元素的字符串集合。

不同之处是有序集合的每个成员都关联了一个评分 (score)，这个评分 (score) 被用来按照从最低分到最高分的方式排序集合中的成员。集合的成员是唯一的，但是评分可以是重复了。

(2) 因为元素是有序的, 所以你也可以很快的根据评分 (score) 或者次序 (position) 来获取一个范围的元素。访问有序集合的中间元素也是非常快的, 因此你能够使用有序集合作为一个没有重复成员的智能列表。

4.6.2 常用操作

1) 往集合中添加指定的 member 及 score

```
127.0.0.1:6379> zadd <key> [<score> <member> ... ]
```

2) 从集合中取出指定下标范围的数据，正序取

```
127.0.0.1:6379> zrange <key> <start> <stop> [WITHSCORES]
```

3) 从集合中取出指定下标范围的数据，倒序取

```
127.0.0.1:6379> zrevrange <key> <start> <stop> [WITHSCORES]
```

4) 从集合中取出指定 score 范围的数据，默认从小到大

```
127.0.0.1:6379> zrangebyscore <key> <min> <max> [WITHSCORES]
```

5) 从集合中取出指定 score 范围的数据，从大到小

```
127.0.0.1:6379> zrevrangebyscore <key> <max> <min> [WITHSCORES]
```

6) 给集合中指定 member 的 score 增加 increment

```
127.0.0.1:6379> zincrby <key> <increment> <member>
```

7) 删除集合中指定的 member

```
127.0.0.1:6379> zrem <key> <member...>
```

8) 统计指定 score 范围的元素个数

```
127.0.0.1:6379> zcount <key> <min> <max>
```

9) 返回集合中指定 member 的排名，排名从 0 开始

```
127.0.0.1:6379> zrank <key> <member>
```

4.6.3 思考: 实现一个文章访问量的排行榜

(1) 使用哪种类型存储?

- (2) key 怎么设计?
- (3) value 怎么设计?
- (4) 写入使用什么命令?
- (5) 查询使用什么命令?

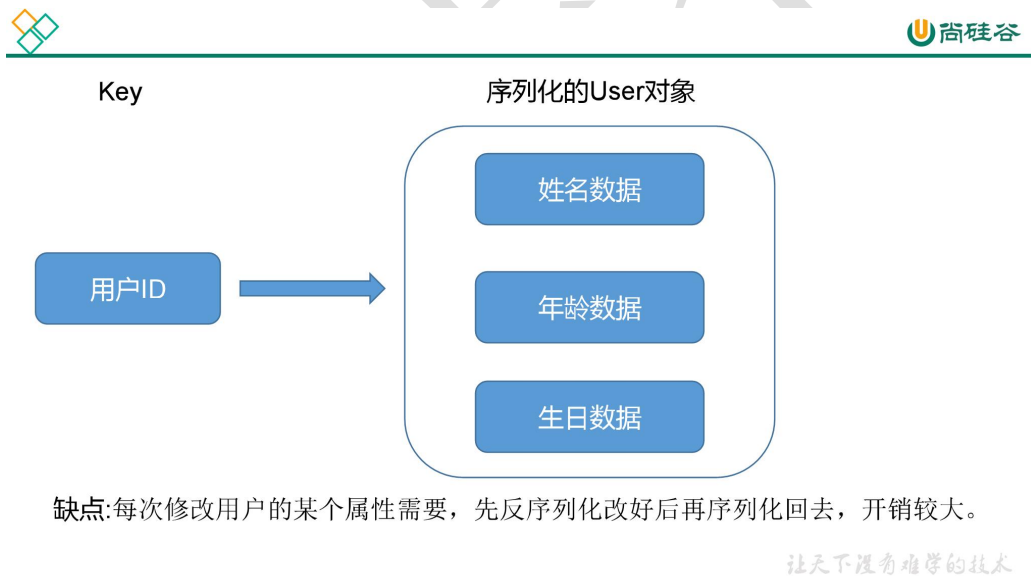
4.7 Hash

4.7.1 特点

- (1) Redis hash 是一个键值对集合
- (2) Redis hash 的值是由多个 field 和 value 组成的映射表
- (3) 类似 Java 里面的 Map<String,Object>

4.7.2 分析一个问题: 现有一个 User 对象, 在 Redis 中如何存?

- 1) 第一种方案: 用户 ID 为 key, VALUE 为 JavaBean 序列化后的字符串

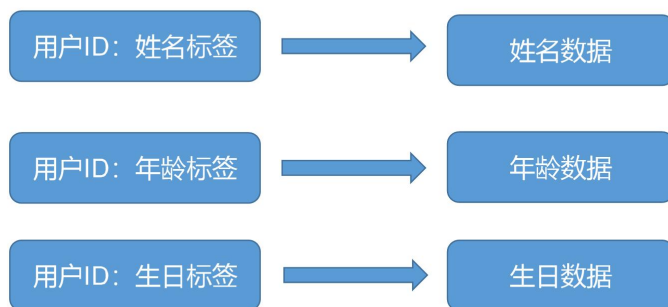


- 2) 第二种方案: 用户 ID+属性名作为 key, 属性值作为 Value.



Key

序列化的User对象



缺点:存储一个用户需要大量的key, 用户ID数据冗余

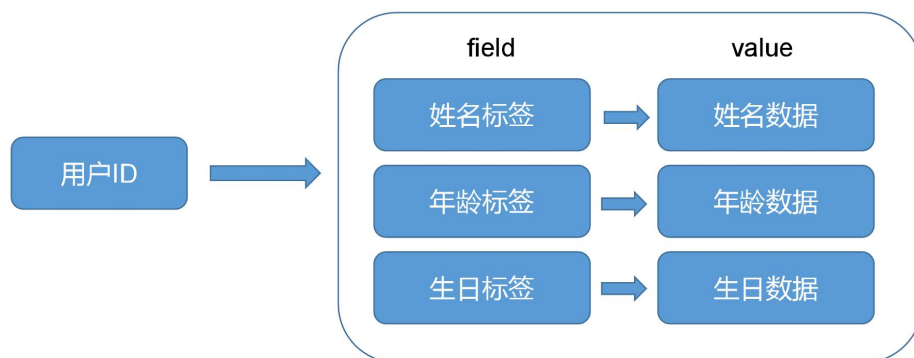
让天下没有难学的技术

3) 第三种方案: 通过 key(用户 ID) + field(属性标签) 就可以操作对应属性数据了, 既不需要重复存储数据, 也不会带来序列化和并发修改控制的问题



Key

hash



让天下没有难学的技术

4.7.3 常用操作

1) 给集合中添加指定的 <field> - <value>

```
127.0.0.1:6379> hset <key> [<field> <value> ...]
```

2) 给集合中添加指定的 <field> - <value>, 当指定的 field 不存在时

```
127.0.0.1:6379> hsetnx <key> <field> <value>
```

3) 取出集合中指定 field 的 value

```
127.0.0.1:6379> hget <key> <field>
```

4) 判断集合中是否存在指定的 field

```
127.0.0.1:6379> hexists <key> <field>
```

5) 列出集合中所有的 field

```
127.0.0.1:6379> hkeys <key>
```

6) 列出集合中所有的 value

```
127.0.0.1:6379> hvals <key>
```

7) 给集合中指定 field 的 value 值增加 increment

```
127.0.0.1:6379> hincrby <key> <field> <increment>
```

第 5 章 Redis 的相关配置

1) 计量单位说明,大小写不敏感

```
# 1k => 1000 bytes
# 1kb => 1024 bytes
# 1m => 1000000 bytes
# 1mb => 1024*1024 bytes
# 1g => 1000000000 bytes
# 1gb => 1024*1024*1024 bytes
#
# units are case insensitive so 1GB 1Gb 1gB are all the same.
```

2) bind

默认情况 bind=127.0.0.1 只能接受本机的访问请求

不写的情况下, 无限制接受任何 ip 地址的访问, 生产环境肯定要写你应用服务器的地址, 如果开启了 protected-mode, 那么在没有设定 bind ip 且没有设密码的情况下, Redis 只允许接受本机的请求

```
#bind 127.0.0.1
protected-mode no
```

3) port 服务端口号

```
port 6379
```

4) daemonize

是否为后台进程

```
daemonize yes
```

5) pidfile

存放 pid 文件的位置, 每个实例会产生一个不同的 pid 文件

```
pidfile /var/run/redis_6379.pid
```

6) log file

日志文件存储位置

```
logfile ""
```

7) database

设定库的数量 默认 16

```
databases 16
```

8) requirepass

设置密码

```
requirepass 123456

127.0.0.1:6379> set k1 v1
(error) NOAUTH Authentication required.
127.0.0.1:6379> auth "123456"
OK
127.0.0.1:6379> set k1 v1
OK
127.0.0.1:6379> get k1
"v1"
```

9) maxmemory

设置 Redis 可以使用的内存量。一旦到达内存使用上限，Redis 将会试图移除内部数据，移除规则可以通过 `maxmemory-policy` 来指定。如果 Redis 无法根据移除规则来移除内存中的数据，或者设置了“不允许移除”，那么 Redis 则会针对那些需要申请内存的指令返回错误信息，比如 SET、LPUSH 等。

```
# maxmemory <bytes>
```

10) maxmemory-policy

移除策略

```
# maxmemory-policy noeviction

#volatile-lru: 使用 LRU 算法移除 key, 只对设置了过期时间的键
#allkeys-lru: 使用 LRU 算法移除 key
#volatile-lfu : 使用 LFU 策略移除 key, 只对设置了过期时间的键.
#allkeys-lfu :使用 LFU 策略移除 key
#volatile-random: 在过期集合中移除随机的 key, 只对设置了过期时间的键
#allkeys-random: 移除随机的 key
#volatile-ttl: 移除那些 TTL 值最小的 key, 即那些最近要过期的 key
#noeviction: 不进行移除。针对写操作，只是返回错误信息
```

11) Maxmemory-samples

设置样本数量，LRU 算法和最小 TTL 算法都并非是精确的算法，而是估算值，所以你可以设置样本的大小。一般设置 3 到 7 的数字，数值越小样本越不准确，但是性能消耗也越小。

```
# maxmemory-samples 5
```

第 6 章 Jedis

Jedis 是 Redis 的 Java 客户端，可以通过 Java 代码的方式操作 Redis

6.1 环境准备

1) 添加依赖

```
<dependency>
  <groupId>redis.clients</groupId>
  <artifactId>jedis</artifactId>
  <version>3.3.0</version>
</dependency>
```

6.2 基本测试

1) 测试连通

```
public class JedisTest {
    public static void main(String[] args) {
        Jedis jedis = new Jedis("hadoop102", 6379);
        String ping = jedis.ping();
        System.out.println(ping);
    }
}
```

2) 连接池

连接池主要用来节省每次连接 redis 服务带来的连接消耗，将连接好的实例反复利用

```
public static JedisPool pool = null ;

public static Jedis getJedis() {
    if (pool == null) {
        // 主 要 配 置
        JedisPoolConfig jedisPoolConfig = new JedisPoolConfig();
        jedisPoolConfig.setMaxTotal(10); // 最大可用连接数
        jedisPoolConfig.setMaxIdle(5); // 最大闲置连接数
        jedisPoolConfig.setMinIdle(5); // 最小闲置连接数
        jedisPoolConfig.setBlockWhenExhausted(true); // 连接耗尽是否等待
        jedisPoolConfig.setMaxWaitMillis(2000); // 等待时间
        jedisPoolConfig.setTestOnBorrow(true); // 取连接的时候进行一下测试
        pool = new JedisPool(jedisPoolConfig, "hadoop102", 6379);
    }

    return pool.getResource();
}

public static void main(String[] args) {
    // Jedis jedis = new Jedis("hadoop202", 6379);
    Jedis jedis = getJedis();
    String ping = jedis.ping();
    System.out.println(ping);
}
```


第 7 章 Redis 持久化

7.1 两种方式

Redis 提供了 2 个不同形式的持久化方式 RDB 和 AOF。

RDB 为快照备份，会在备份时将内存中的所有数据持久化到磁盘的一个文件中。

AOF 为日志备份，会将所有写操作命令记录在一个日志文件中。

7.2 RDB (Redis Database)

7.2.1 是什么

在指定的时间间隔内将内存中的数据快照写入磁盘，也就是行话讲的 Snapshot 快照，它恢复时是将快照文件直接读到内存里。

7.2.2 如何执行持久化

Redis 会单独创建 (fork) 一个子进程来进行持久化，会先将数据写入到一个临时文件中，待持久化过程都结束了，再用这个临时文件替换上次持久化好的文件。整个过程中，主进程是不进行任何 IO 操作的，这就确保了极高的性能。如果需要进行大规模数据的恢复，且对于数据恢复的完整性不是非常敏感，那 RDB 方式要比 AOF 方式更加的高效。RDB 的缺点是最后一次持久化后的数据可能丢失。

7.2.3 RDB 文件

1) RDB 保存的文件

在 redis.conf 中配置文件名称，默认为 dump.rdb

```
dbfilename dump.rdb
```

2) RDB 文件的保存路径

默认为 Redis 启动时命令行所在的目录下,也可以修改

```
dir ./
```

7.2.4 RDB 保存策略

```
# save <seconds> <changes>

# Unless specified otherwise, by default Redis will save the DB:
# * After 3600 seconds (an hour) if at least 1 key changed
# * After 300 seconds (5 minutes) if at least 100 keys changed
# * After 60 seconds if at least 10000 keys changed
#
```

17

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
# You can set these explicitly by uncommenting the three
following lines.
#
# save 3600 1
# save 300 100
# save 60 10000
```

7.2.5 手动保存

- (1) save: 只管保存, 其它不管, 全部阻塞
- (2) bgsave: 按照保存策略自动保存
- (3) shutdown 时服务会立刻执行备份后再关闭
- (4) flushall 时会将清空后的数据备份

7.2.6 RDB 备份恢复

1) 备份

将 dump.rdb 文件拷贝到要备份的位置

2) 恢复

关闭 Redis, 把备份的文件拷贝到工作目录下, 启动 redis, 备份数据会直接加载。

7.2.7 RDB 其他配置

1) 进行 rdb 保存时, 将文件压缩

```
rdbcompression yes
```

2) 文件校验

在存储快照后, 还可以让 Redis 使用 CRC64 算法来进行数据校验, 但是这样做会增加大约 10% 的性能消耗, 如果希望获取到最大的性能提升, 可以关闭此功能

```
rdbchecksum yes
```

7.2.8 RDB 优缺点

1) 优点:

节省磁盘空间, 恢复速度快.

2) 缺点:

虽然 Redis 在 fork 时使用了写时拷贝技术, 但是如果数据庞大时还是比较消耗性能。在备份周期在一定间隔时间做一次备份, 所以如果 Redis 意外 down 掉的话, 就会丢失最后一次快照后的所有修改

7.3 AOF (Append Only File)

7.3.1 是什么

以日志的形式来记录每个写操作，将 Redis 执行过的所有写指令记录下来(读操作不记录)，只许追加文件但不可以改写文件，Redis 启动之初会读取该文件重新构建数据，换言之，Redis 重启的话就根据日志文件的内容将写指令从前到后执行一次以完成数据的恢复工作。

7.3.2 开启 AOF

1) AOF 默认不开启，需要手动在配置文件中配置

```
appendonly no
```

2) AOF 文件

```
appendfilename "appendonly.aof"
```

3) AOF 文件保存的位置，与 RDB 的路径一致

```
dir ./
```

7.3.3 AOF 同步频率

```
# no: don't fsync, just let the OS flush the data when it wants.
Faster.
# always: fsync after every write to the append only log. Slow,
Safest.
# everysec: fsync only one time every second. Compromise.
```

7.3.4 AOF 文件损坏恢复

```
redis-check-aof --fix appendonly.aof
```

7.3.5 AOF 备份

AOF 的备份机制和性能虽然和 RDB 不同，但是备份和恢复的操作同 RDB 一样，都是拷贝备份文件，需要恢复时再拷贝到 Redis 工作目录下，启动系统即加载

7.3.6 Rewrite

AOF 采用文件追加方式，文件会越来越大为避免出现此种情况，新增了重写机制，当 AOF 文件的大小超过所设定的阈值时，Redis 就会启动 AOF 文件的重写，只保留可以恢复数据的最小指令集。可以使用命令 `bgrewriteaof` 手动开始重写。

重写虽然可以节约大量磁盘空间，减少恢复时间。但是每次重写还是有一定的负担的，因此设定 Redis 要满足一定条件才会进行重写。

系统载入时或者上次重写完毕时，Redis 会记录此时 AOF 大小，设为 `base_size`，如果 Redis 的 AOF 当前大小 $\geq \text{base_size} + \text{base_size} * 100\%$ (默认) 且当前大小 $\geq 64\text{mb}$ (默认) 的情况下，Redis 会对 AOF 进行重写。

```
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
```

7.3.7 AOF 的优缺点

1) 优点:

- (1) 备份机制更稳健，丢失数据概率更低。
- (2) 可读的日志文本，通过操作 AOF 文件，可以处理误操作。

2) 缺点:

- (1) 比起 RDB 占用更多的磁盘空间
- (2) 恢复备份速度要慢
- (3) 每次写都同步的话，有一定的性能压力
- (4) 存在个别 bug，造成恢复不能

7.4 持久化的优先级

AOF 的优先级大于 RDB，如果同时开启了 AOF 和 RDB，Redis 服务启动时恢复数据以 AOF 为准。

7.5 RDB 和 AOF 用哪个好

- (1) 官方推荐两个都启用。
- (2) 如果对数据完整性不敏感，可以选单独用 RDB
- (3) 不建议单独用 AOF，因为可能会出现 Bug。
- (4) 如果只是做纯内存缓存，可以都不用